數位邏輯系統設計 作業十

第十一章 電子鐘專題

通訊 4A 0106c033 桑瑋佑

一 實驗目的:

本章實驗的目標，是透過 LED 數字顯示的控制，逐步建立一個數字式的電子鐘。利用原來計數和顯示的基本結構，利用計數器的 CLR 復歸訊號和 EC 時鐘，加以擴張使用到每一個數字的預設和歸零控制。

二 步驟:

實驗時，先用 LoadPGA 程式將 ch11.rbt 的電路檔載入 FPGA 中，依定義來控制按鍵，分別實現 1.歸零 2.換位 3.設定完後開始執行等動作。

1. 利用 YC1 來選擇四位數字中的一位，每按一次就右移一位數字，全部數字移完後就停止；再按一次又重新開始。
2. 利用 YC0 來調整時間，每次按下時就所選擇的數字就會連續上跳，跳完 9 再回 到 0 繼續循環。
3. 目前選到的數字就會閃動，這樣操作者就能確定現在選擇的數字了。
4. 當 YC2 按鍵按下時計時開始，再按一次計時停止，再按一次又繼續。
5. 當 YC3 按鍵按下時計時歸零。

三 結果:以下為程式碼，為 CH11.vhd 檔

```
--The IEEE standard 1164 package, declares std_logic, rising_edge(), etc.
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_arith.all;
use IEEE.std_logic_unsigned.all;
```

```
--
--*********************************************************
-- CH11C : digital clock with 7-segment LED control (test #C)
--                                designed by Pei-Chong Tang, Jan. 1999
--*********************************************************
--
entity CH11C is
        port
        (       CLK : in    STD_LOGIC;                                  --4MHz clock
                XA    : out STD_LOGIC_VECTOR (3 downto 0);    --output port
                XB    : out STD_LOGIC_VECTOR (7 downto 0);    --input/output
                XC    : in    STD_LOGIC_VECTOR (3 downto 0)      --input    port
        );
end CH11C;

architecture CH11C_ARCH of CH11C is
        component DIGIT10       --***** one digit with divide-by-10 *****
        port
        (       CLK : in        STD_LOGIC;                                  --system clock
                DB    : out      STD_LOGIC_VECTOR (3 downto 0); --digit bus
                ENB : in        STD_LOGIC;                                  --output enable
                CLR : in        STD_LOGIC;                                  --clear signal
                EC    : in        STD_LOGIC;                                  --carry in
                CY    : out      STD_LOGIC                                   --carry out
        );
        end component;
        component DIGIT6         --***** one digit with divide-by-6    *****
        port
        (       CLK : in        STD_LOGIC;                                  --system clock
                DB    : out      STD_LOGIC_VECTOR (3 downto 0); --digit bus
                ENB : in        STD_LOGIC;                                  --output enable
                CLR : in        STD_LOGIC;                                  --clear signal
                EC    : in        STD_LOGIC;                                  --carry in
                CY    : out      STD_LOGIC                                   --carry out
        );
        end component;
                                --***** global signals *****
```

```
        signal DB    : STD_LOGIC_VECTOR (3 downto 0);
        signal SEG : STD_LOGIC_VECTOR (6 downto 0);
        signal ENB : STD_LOGIC_VECTOR (3 downto 0);
        signal HZ,SEC,CLR,HOLD : STD_LOGIC;
        signal SAMPLE,MATCH        : STD_LOGIC;
        signal KEY,FLT,DIF          : STD_LOGIC;
        signal SEL : STD_LOGIC_VECTOR (2 downto 0);
begin
--
--*********************************************************
--
-- system connection for the experiment
--
SYSTEM_CONNECT : block
        signal EC, EC1,EC2,EC3,EC4 : STD_LOGIC;
        signal CY0,CY1,CY2,CY3,CY4 : STD_LOGIC;
begin
        U1: DIGIT10    port map (CLK,DB,ENB(0),CLR,EC1,CY1);
        U2: DIGIT6     port map (CLK,DB,ENB(1),CLR,EC2,CY2);
        U3: DIGIT10    port map (CLK,DB,ENB(2),CLR,EC3,CY3);
        U4: DIGIT6     port map (CLK,DB,ENB(3),CLR,EC4,CY4);
        CLR <= not XC(3);
        EC    <=    HZ and not SEL(2) and not XC(0);
        CY0 <=    HZ and not HOLD;
        EC1 <= (CY0 and SEL(2)) or (EC and not SEL(1) and not SEL(0));
        EC2 <= (CY1 and SEL(2)) or (EC and not SEL(1) and          SEL(0));
        EC3 <= (CY2 and SEL(2)) or (EC and          SEL(1) and not SEL(0));
        EC4 <= (CY3 and SEL(2)) or (EC and          SEL(1) and          SEL(0));
        KEY <= not XC(0) or not XC(1) or not XC(2) or not XC(3);
        XA    <= not ENB;
        XB(7) <= SEC and ENB(2) and SEL(2) and not HOLD;
GEN: for I in 0 to 6 generate
        XB(I) <= SEG(I) and (SEC or not MATCH or SEL(2));
        end generate;
end block SYSTEM_CONNECT;
--
--*********************************************************
-- 24-bit free counter
```

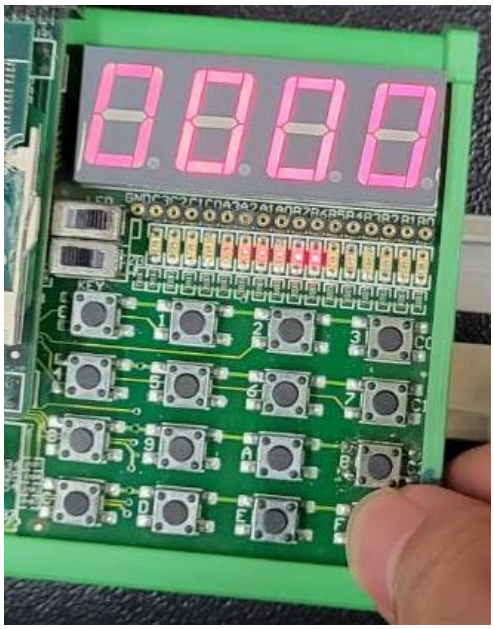
```
--
FREE_COUNTER : block
        signal Q : STD_LOGIC_VECTOR (23 downto 0);
        signal D : STD_LOGIC_VECTOR (1    downto 0);
        signal DLY1,DLY2 : STD_LOGIC;
begin
        process (CLK)
        begin
                if CLK'event and CLK='1' then
                        DLY1 <= Q(23);
                        DLY2 <= Q(15);
                        Q    <= Q+1;
                end if;
        end process;
        SEC       <= Q(23);
        HZ        <= Q(23) and not DLY1;
        SAMPLE <= Q(15) and not DLY2;
        D <= Q(17 downto 16);
        ENB <= "0001" when D=0 else
                "0010" when D=1 else
                "0100" when D=2 else
                "1000" when D=3 else
                "0000";
        MATCH <= '1' when D=SEL(1 downto 0) else '0';
end block FREE_COUNTER;
--
--*********************************************************
--
-- keyboard debouncing
--
DEBOUNCING : block
        signal D0,D1 : STD_LOGIC;
begin
        process (CLK)
        begin
                if CLK'event and CLK='1' then        --CLK rising
                        if SAMPLE='1' then                --CLK enable
                                D1<=D0; D0<=KEY;             --delay
```

```
                        FLT <= ((D0 and D1) or FLT) and (D0 or D1); --RS-F/F
                end if;
            end if;
        end process;
end block DEBOUNCING;
--
--*********************************************************
--
-- differential signal
--
DIFFERENTIAL : block
        signal D0,D1 : STD_LOGIC;
begin
        process (CLK)
        begin
            if CLK'event and CLK='1' then        --CLK rising
                D1<=D0; D0<=FLT;                     --delay
            end if;
        end process;
        DIF <= D0 and not D1;                        --differential
end block DIFFERENTIAL;
--
--*********************************************************
-- timer control
--
TIMER_CONTROL : block
        signal SET,EC : STD_LOGIC;
begin
        process (CLK)
        begin
            if SET='1' then
                HOLD <= '1';
            elsif CLK'event and CLK='1' then
                if EC='1' then
                    HOLD <= not HOLD;
                end if;
            end if;
        end process;
```

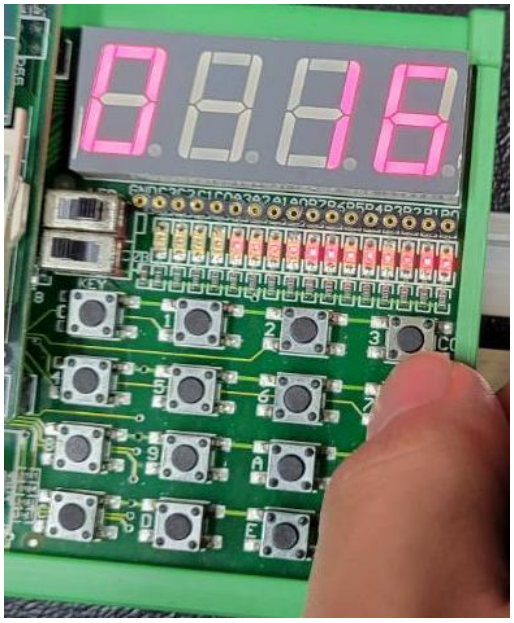
```
        SET <= not XC(3);
        EC    <= not XC(2) and DIF;
end block TIMER_CONTROL;
--
--*********************************************************
-- digit select
--
DIGIT_SELECT : block
        signal Q : STD_LOGIC_VECTOR (2 downto 0);
        signal SET,EC : STD_LOGIC;
begin
        process (CLK)
        begin
                if SET='1' then
                        Q <= "100";
                elsif CLK'event and CLK='1' then
                        if EC='1' then
                                Q <= Q-1;
                        end if;
                end if;
        end process;
        SET <= '1' when Q=7 else '0';
        EC    <= DIF and not XC(1);
        SEL <= Q;
end block DIGIT_SELECT;
--
--*********************************************************
-- seven-segment LED transformation
--
SEVEN_SEGMENT : block
begin
                --GFEDCBA
        SEG <= "0111111" when DB=0    else
                "0000110" when DB=1    else
                "1011011" when DB=2    else
                "1001111" when DB=3    else
                "1100110" when DB=4    else
                "1101101" when DB=5    else
```

```
                    "1111101" when DB=6    else
                    "0000111" when DB=7    else
                    "1111111" when DB=8    else
                    "1101111" when DB=9    else
                    "0000000";
end block SEVEN_SEGMENT;
--
--*****************************************************
-- end of architechture
--
end CH11C_ARCH;
```

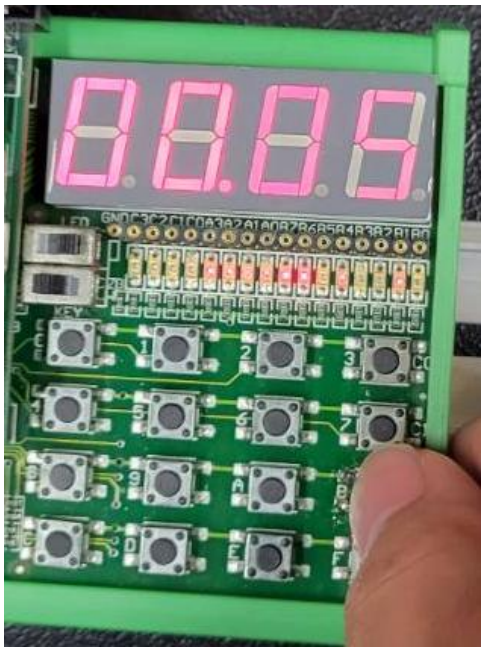
以下為成果圖



此為按下 XC3，會執行歸零。



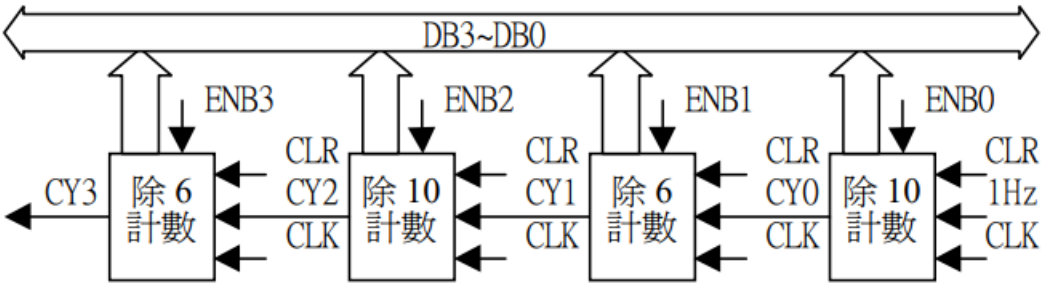
此為按下 XC1，會執行換位，選到的數字就會閃爍。



此為按下 XC2，按下後開始計時，再按一次則停止。

四 重點提醒

1.



其中:

1. 所有 CLK 共用系統時鐘訊號，建立同步結構基準。
2. 每一級的 CY 進位送到下一級的 EC 輸入。
3. 在任何一瞬間，ENB3~ENB0 中只有一組准許動作，相當於多選一的選擇電路送 到匯流排 DB3~DB0 中。當 ENB3~ENB0 輪流動作時，就造成了 LED 矩陣掃瞄的同步效應。
4. 所有 CLR 訊號一起作動，造成時鐘歸零的作用。

2.除六計數器和除十計數器的差異:

1. 除 10 計數器逢 10 復歸，進位點為 1001->0000 的瞬間，因此我們用 Q3 的負向微分作為進位依據。
2. 除 6 計數器逢 6 復歸，進位點為 0101->0000 的瞬間，因此我們改用 Q2 的負向微分作為進位依據。

3.LED 閃動控制:其工作原則如下

1. 將原來直接送到 XB 輸出的七段式 SEG 訊號，現在要 and 某種條件後才送出；如果條件不符就會讓 XB=0 而熄滅 LED 的顯示。
2. 條件包括下面三項：
   a. 當選擇設定模式時才閃亮，這部分由 SEL(2)訊號決定；
   b. 當掃瞄到指定數字時才閃亮，這部分由 MATCH 訊號決定；
   c. 每秒閃動一次，這部分由 SEC 訊號決定。

五 心得

這次的程式碼非常的長，而且應該算是比較完整的專題內容，電子鐘在我們生活中隨處可見，沒想到在實現的過程中，藏著這麼多知識，為了實現這次電子鐘專題，結合前面學過非常多的程式應用，在練習活用之餘也是讓我對於以前的內容越來越熟悉，可以比較快速反應那些情況會需要用到那些程式碼，好好把前面的基本弄熟，在後半段要活用的時候，就不會突然變很難，很多都是以前學習過的，底子打穩後，再來活用反而是有趣的事情。